DFM — Defect Management Module

DFM — Defect Management Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Quality Standard (SQS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Defect Management Governance

Module ID: SLGA-SQS-DFM-004

Origin ID: OOF.CM.SLGA.05.04.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 12.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Defect Management Module (DFM) governs the identification,

recognition, classification, assessment, attribution, prioritization, correction reference, resolution verification, evidence integrity, and lifecycle continuity of software defects.

DFM establishes the methodological conditions under which an observed

software deviation can be recognized as an identifiable, evidence-supported defect, associated with the relevant software artifact and operating context, assessed according to its material consequences, and subsequently recognized as resolved only when competent verification evidence supports that determination.

It governs the relationships among expected software behavior, observed

deviations, defect reports, reproduction conditions, affected artifacts, defect severity, operational impact, root-cause findings, corrective actions, verification evidence, regression implications, residual limitations, and defect closure

decisions.

DFM distinguishes test failure from verified software defect, defect

identification from root-cause attribution, correction implementation from verified resolution, administrative closure from technical resolution, and defect resolution in one configuration from demonstrated resolution across all relevant configurations.

It applies to software defects identified or managed by humans,

organizations, automated testing systems, AI coding assistants, autonomous development agents, and software infrastructure mechanisms.

DFM is technology-independent and does not prescribe a particular issue-tracking platform, programming language, debugging tool, development methodology, or defect remediation process.

Its principal output is:

Governed Software Defect State

Its foundational distinction is:

Defect Closure ≠ Verified Defect Resolution.

Operational Role

The operational role of DFM is:

To ensure that software defects remain identifiable, attributable only to the extent supported by evidence, assessed according to their actual or reasonably supported significance, connected to the software artifacts and operating conditions affected, and recognized as resolved only through competent verification against the relevant correction and software state.

DFM establishes the defect-governance mechanism of SQS.

It determines what deviation has been identified, whether the available

evidence supports recognizing it as a defect, which software conditions are affected, what correction has been proposed or implemented, and whether the claimed resolution has been sufficiently demonstrated.

DFM does not independently authorize software changes, establish testing criteria, accept residual quality risk, authorize software release, or determine legal liability.

Operational Question

How can an observed software deviation be competently recognized, classified, assessed, traced to the affected software state, and verified as resolved without confusing reported failures, proposed corrections, successful test execution, or administrative ticket closure with demonstrated defect resolution?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

DFM operates within:

Software Quality Standard (SQS)

The SQS internal governance flow is:

Quality Requirements → Quality Criteria Definition → Testing & Coverage

Management → Defect Management → Quality Conformity & Acceptance

DFM governs:

Defect Management

It is the fourth Core Module of SQS.

Its principal upstream references include:

Governed Software Quality Requirements State

from QRM — Quality Requirements Module.

Governed Quality Criteria Definition State

from QCDM — Quality Criteria Definition Module.

Governed Software Testing & Coverage State

from TCMM — Testing & Coverage Management Module.

Relevant supporting references include:

Governed Software Creation State

from Software Creation Standard (SCS).

Governed Software Dependency State

from Software Dependency Standard (SDS).

Governed Software Configuration State

from Software Configuration Standard (SCFS).

Its principal output is:

Governed Software Defect State

This output supports:

QCAM — Quality Conformity & Acceptance Module

and contributes to:

Governed Software Quality State

DFM may be invoked whenever defects are reported, identified, reassessed, corrected, verified, reopened, superseded, or evaluated for downstream quality acceptance.

• Governed Space

Software Defect Management Governance

This space governs the conditions under which software defects become

identifiable, evidence-supported, appropriately assessed, and verifiably resolved.

Its principal dimensions include:

• Defect governance object identity

• Defect report origin

• Defect observation

• Expected behavior

• Observed behavior

• Expected–observed deviation

• Defect recognition

• Defect confirmation

• Defect classification

• Defect scope

• Affected software object

• Artifact identity

• Artifact version

• Configuration correspondence

• Environment correspondence

• Reproduction conditions

• Reproducibility

• Intermittent defects

• Defect severity

• Defect priority

• Defect impact

• Defect criticality

• Defect exposure

• Defect propagation

• Root-cause investigation

• Root-cause evidence

• Defect attribution

• Defect dependencies

• Defect ownership references

• Defect correction proposal

• Correction authority references

• Correction implementation references

• Corrected artifact correspondence

• Defect resolution criteria

• Resolution verification

• Regression verification

• Verification evidence

• Verification competence

• Verification integrity

• Defect closure

• Conditional closure

• Defect reopening

• Residual defect conditions

• Known defect limitations

• Defect uncertainty

• Defect conflict

• Defect evidence gaps

• Critical defect gaps

• Defect history

• Defect continuity

• Defect reassessment

• Defect governance states

This governed space remains distinct from test execution under TCMM,

software change authorization under SCMS, and quality acceptance under QCAM.

Defect Governance Object

Every material DFM assessment shall establish a Defect Governance Object.

The object represents an identifiable software defect or candidate defect

and its governance conditions.

It should identify:

Defect Identity: The identifiable defect reference.

Report Origin: The source of the report.

Observed Deviation: The behavior requiring assessment.

Expected Behavior: The applicable expected condition.

Software Object: The affected software.

Artifact Reference: The relevant software artifact.

Configuration: The applicable software configuration.

Environment: The conditions under which the deviation occurred.

Recognition State: Whether the deviation is confirmed as a defect.

Severity: The assessed technical significance.

Impact: The relevant operational consequences.

Correction Reference: The proposed or implemented correction.

Verification Evidence: The evidence supporting resolution.

Defect State: The recognized governance condition.

A Defect Governance Object may exist before defect confirmation.

Defect Report ≠ Confirmed Software Defect.

Defect Report Origin

DFM shall identify the origin of material defect reports.

Origins may include:

• Automated testing

• Human testing

• Production monitoring

• User reports

• Developer observations

• AI code analysis

• Static analysis

• Integration assessment

• Operational incident investigation

Report origin shall remain distinguishable from defect confirmation.

Defect Report Source ≠ Defect Validity.

Defect Observation Governance

DFM shall preserve the actual observation giving rise to a defect claim.

Observations may include:

• Incorrect output

• Unexpected behavior

• Application failure

• Performance deviation

• Data processing error

• Integration malfunction

• Recovery failure

• Inconsistent behavior

Observations shall remain connected to the relevant software and

operational conditions.

Observed Software Anomaly ≠ Confirmed Software Defect.

Expected Behavior Correspondence

DFM shall identify the expected behavior against which a reported deviation

is assessed.

Expected behavior should derive from:

• Governed quality requirements

• Governed quality criteria

• Applicable software specifications

• Competent behavioral references

Where expected behavior remains ambiguous, defect confirmation may remain unresolved.

Unexpected Behavior ≠ Defect Without an Applicable Expected Condition.

Observed Behavior Correspondence

DFM shall preserve the behavior actually observed.

The record should identify:

• Relevant inputs

• Execution conditions

• Actual outputs

• Error conditions

• State transitions

• Relevant timing

• Available supporting evidence

Observed behavior shall not be replaced by inferred or assumed behavior.

Reported Behavior ≠ Independently Verified Behavior.

Expected–Observed Deviation

DFM shall establish whether a material deviation exists between expected

and observed behavior.

Possible determinations include:

• Confirmed deviation

• Partially supported deviation

• Inconclusive deviation

• Invalid comparison

• No demonstrated deviation

Deviation recognition shall remain bounded by the applicable evidence.

Observed Deviation ≠ Established Root Cause.

Defect Recognition

DFM shall determine whether available evidence supports recognizing an

observed deviation as a software defect.

Recognition may require:

• Applicable expected behavior

• Competent observation

• Relevant software correspondence

• Sufficient deviation evidence

• Exclusion or recognition of alternative explanations

A failed test alone shall not automatically establish a software defect.

Failed Test ≠ Confirmed Software Defect.

Defect Confirmation

DFM shall distinguish candidate defects from sufficiently confirmed defects.

Confirmation may depend on:

• Reproduction

• Competent diagnostic evidence

• Consistent observed deviation

• Verified software correspondence

• Applicable quality criteria

Reproduction is useful but shall not be an absolute prerequisite where other competent evidence sufficiently establishes the defect.

Non-Reproducible Defect ≠ Nonexistent Defect.

Defect Classification

DFM shall classify material defects according to their nature.

Classes may include:

• Functional defect

• Reliability defect

• Performance defect

• Compatibility defect

• Usability defect

• Maintainability-related defect

• Portability defect

• Recoverability defect

• Integration defect

• Configuration-dependent defect

Classification shall remain distinguishable from severity.

Defect Classification ≠ Defect Severity.

Defect Scope

DFM shall identify the known or reasonably supported scope of a defect.

Scope may concern:

• Specific function

• Component

• Application

• Integration

• Software version

• Configuration

• Environment

• User group

• Operational scenario

Unknown scope shall remain explicit.

Observed Defect Scope ≠ Complete Defect Scope.

Affected Software Object

DFM shall identify the software object affected by the defect.

Objects may include:

• Function

• Component

• Service

• API

• Library

• Application

• Embedded system

• Integrated software system

The affected object shall remain distinguishable from the object

responsible for the underlying cause.

Affected Software Component ≠ Root-Cause Component.

Artifact Identity Governance

DFM shall preserve the identity of software artifacts associated with

defect observations and corrections.

Relevant references may include:

• Artifact identifier

• Version

• Build reference

• Content integrity reference

• Relevant configuration

• Correction artifact

A version label alone shall not establish artifact correspondence.

Declared Defective Version ≠ Verified Defective Artifact.

Artifact Version Correspondence

DFM shall identify the software versions in which a defect has been

observed, confirmed, corrected, or verified.

A defect confirmed in one version shall not automatically be assumed

present or absent in another.

Defect in Version A ≠ Proven Defect in Version B.

Configuration Correspondence

DFM shall preserve relevant configuration conditions associated with defect

behavior.

Conditions may include:

• Runtime parameters

• Feature flags

• Dependency versions

• Environment variables

• Build settings

• Integration configuration

Configuration changes may affect defect occurrence and correction validity.

Defect Resolved in One Configuration ≠ Defect Resolved in Every

Configuration.

Environment Correspondence

DFM shall identify relevant environmental conditions associated with defect

observations.

Conditions may include:

• Operating system

• Hardware

• Runtime

• Network

• Infrastructure

• External services

• Workload

• Data characteristics

A defect may be environment-dependent.

Environment-Specific Defect ≠ Universal Software Defect.

Defect Reproduction Conditions

DFM shall preserve known conditions under which a defect can be reproduced.

Conditions may include:

• Input values

• Execution sequence

• Software state

• Configuration

• Environment

• Timing

• Dependency behavior

• Workload

Reproduction conditions shall remain associated with the relevant artifact.

Documented Reproduction Steps ≠ Successfully Reproduced Defect.

Defect Reproducibility

DFM shall assess whether the reported defect can be reproduced under sufficiently comparable conditions.

Possible conditions include:

• Consistently reproducible

• Intermittently reproducible

• Reproducible only under specific conditions

• Not yet reproduced

• Reproduction evidence unavailable

Lack of reproduction shall not automatically invalidate competent

observational evidence.

Failure to Reproduce ≠ Proof of Defect Absence.

Intermittent Defect Governance

DFM shall recognize defects that occur inconsistently.

Intermittent defects may arise from:

• Concurrency

• Timing

• Resource contention

• Distributed execution

• External dependencies

• Data ordering

• Environmental variation

Intermittent behavior shall not automatically be classified as a test

framework defect.

Intermittent Failure ≠ Automatically Invalid Defect Evidence.

Defect Severity Governance

DFM shall assess the technical significance of a defect.

Severity may concern:

• Functional disruption

• Data correctness

• Reliability degradation

• Performance impact

• Recovery limitations

• Operational consequences

Severity shall remain supported by the known defect conditions.

Defect Severity ≠ Defect Priority.

Defect Priority Governance

DFM shall recognize the priority assigned to defect investigation or

correction.

Priority may depend on:

• Severity

• Operational urgency

• User impact

• Available workarounds

• Release objectives

• Resource constraints

• Relevant dependencies

Priority shall remain distinguishable from the technical significance of

the defect.

High Defect Priority ≠ High Technical Severity.

Defect Impact Governance

DFM shall assess the actual or reasonably supported impact of material

defects.

Impact may include:

• Incorrect processing

• Operational disruption

• User task failure

• Service degradation

• Data inconsistency

• Recovery difficulty

• Dependency effects

Actual impact shall remain distinguishable from hypothetical impact.

Potential Defect Impact ≠ Observed Defect Impact.

Defect Criticality

DFM shall identify defects whose consequences are materially significant within the relevant software context.

Criticality may depend on:

• Intended use

• Affected functionality

• Operational consequences

• Recovery limitations

• Exposure

• Dependency propagation

• Available mitigation

Criticality shall remain context-dependent.

Critical Defect Classification ≠ Automatic Release Rejection.

Defect Exposure

DFM shall identify conditions affecting the likelihood or extent of

exposure to a defect.

Exposure may concern:

• User population

• Feature usage

• Workload

• Input frequency

• Deployment scope

• Relevant configuration

• Operating duration

Exposure estimates shall preserve uncertainty.

Defect Exposure Estimate ≠ Verified Defect Occurrence Frequency.

Defect Propagation

DFM shall recognize the possibility that a defect affects additional software functions, components, or dependent systems.

Propagation may involve:

• Shared libraries

• Common data structures

• Integration interfaces

• Reused algorithms

• Configuration dependencies

• Downstream processing

Possible propagation shall remain distinguishable from verified propagation.

Potential Defect Propagation ≠ Demonstrated Propagation.

Root-Cause Investigation

DFM shall preserve the investigation of causes associated with material

defects.

Investigation may involve:

• Execution traces

• Source-code analysis

• Configuration analysis

• Dependency assessment

• Data examination

• Reproduction

• Change history

• Environment analysis

Root-cause investigation shall not be represented as completed merely

because a plausible explanation has been proposed.

Plausible Root Cause ≠ Verified Root Cause.

Root-Cause Evidence

DFM shall assess the competence of evidence supporting a root-cause

determination.

Evidence may include:

• Reproducible causal behavior

• Controlled comparisons

• Code-path evidence

• Relevant logs

• Configuration correspondence

• Dependency behavior

• Corrective experiment results

A correlation between a code change and defect disappearance shall not

automatically establish complete causal attribution.

Correlation with Defect Disappearance ≠ Complete Root-Cause Proof.

Defect Attribution

DFM shall preserve the attribution of defects to relevant technical

conditions only to the extent supported by evidence.

Attribution may concern:

• Software logic

• Component behavior

• Configuration

• Dependency interaction

• Integration conditions

• Data processing

• Environmental factors

Technical defect attribution shall remain distinct from personal

responsibility and legal liability.

Technical Defect Attribution ≠ Legal Liability Attribution.

Defect Dependency Relationships

DFM shall identify material relationships between defects and relevant

dependencies.

Relationships may involve:

• Shared components

• External libraries

• Runtime behavior

• Infrastructure services

• Interface contracts

• Configuration dependencies

Dependency identification shall not independently establish causation.

Dependency Associated with Defect ≠ Dependency Proven as Root Cause.

Defect Ownership References

DFM shall preserve relevant responsibility references for defect

investigation, correction, and verification.

These may identify:

• Responsible development team

• Component maintainer

• Assigned investigator

• Correction implementer

• Verification function

• Competent closure authority

An assignment shall not independently establish capability, authority, or

actual control.

Assigned Defect Owner ≠ Demonstrated Defect Resolution Capability.

Defect Correction Proposal

DFM shall recognize proposed corrective actions.

Proposals may include:

• Code modification

• Configuration correction

• Dependency update

• Algorithm revision

• Input validation change

• Integration correction

• Error-handling improvement

A proposed correction shall remain distinguishable from an implemented

correction.

Proposed Defect Correction ≠ Implemented Defect Correction.

Correction Authority References

DFM shall preserve the authority references applicable to material

corrective actions.

Relevant authority may concern:

• Source-code modification

• Configuration changes

• Dependency replacement

• Production intervention

• Automated agent actions

• Approval requirements

DFM records relevant authority conditions but does not replace SCS, SCFS, SCMS, or agent-governance authority mechanisms.

Authority to Diagnose a Defect ≠ Authority to Modify Production Software.

Correction Implementation References

DFM shall preserve identifiable references to implemented corrections.

References may include:

• Code change

• Commit

• Change request

• Build artifact

• Configuration revision

• Dependency update

• Relevant implementation evidence

Implementation evidence shall not independently establish resolution.

Correction Implemented ≠ Defect Resolved.

Corrected Artifact Correspondence

DFM shall identify the actual software artifact containing the proposed

correction.

Correspondence should establish:

• Corrected artifact identity

• Relevant version

• Build reference

• Configuration

• Correction inclusion

• Relationship to the original defective artifact

A correction present in a development branch shall not automatically

establish its presence in a release candidate.

Correction in Source Code ≠ Correction in Relevant Executable Artifact.

Defect Resolution Criteria

DFM shall identify the conditions under which a defect may be recognized as resolved.

Resolution criteria may require:

• Corrected behavior

• Relevant artifact correspondence

• Suitable verification method

• Applicable environment

• Reproduction-condition assessment

• Required regression evidence

• Absence of the original deviation within the assessed scope

Resolution criteria shall remain connected to the defect actually

recognized.

Defined Resolution Criteria ≠ Demonstrated Resolution.

Defect Resolution Verification

DFM shall require competent evidence before recognizing a material defect

as technically resolved.

Verification may involve:

• Re-execution of relevant tests

• Controlled reproduction comparison

• Verification against corrected artifacts

• Confirmation of applicable configuration

• Observation of corrected behavior

• Assessment of residual limitations

Resolution verification shall not rely solely on developer declarations or

administrative ticket status.

Defect Closure ≠ Verified Defect Resolution.

Verification Method Suitability

DFM shall assess whether the selected verification method can meaningfully establish the claimed defect resolution.

For example:

A unit test may demonstrate correction of a specific function.

It may be insufficient to verify a defect that occurs only under concurrent production workloads.

A mocked integration test may demonstrate local behavior.

It may be insufficient to verify an actual external integration defect.

Successful Verification Method Execution ≠ Suitable Resolution

Verification.

Verification Artifact Identity

DFM shall establish the identity of the software artifact against which

resolution verification was performed.

The verification artifact shall be connected to the implemented correction.

Where correspondence is unresolved, the resolution claim shall remain evidence-limited.

Verification Against an Unidentified Artifact ≠ Verified Defect

Resolution.

Verification Configuration Correspondence

DFM shall preserve the configuration under which defect resolution was

verified.

Where the defect depends on configuration, verification shall address the relevant configuration conditions.

A passing verification in one configuration shall not automatically

establish resolution in another.

Verified Correction in One Configuration ≠ Universal Defect Resolution.

Verification Environment Correspondence

DFM shall assess whether the verification environment is sufficiently representative of the conditions relevant to the defect.

Relevant conditions may include:

• Hardware

• Runtime

• Workload

• Network

• External services

• Data

• Integration conditions

Material differences shall remain explicit.

Verification Environment Similarity ≠ Verified Operational Equivalence.

Defect Regression Verification

DFM shall recognize the need to assess whether corrective changes introduce

or expose additional software deviations.

Regression verification may concern:

• Related functions

• Shared components

• Integration behavior

• Configuration interactions

• Relevant historical defects

• Previously established quality conditions

TCMM governs actual regression test execution and coverage evidence.

DFM evaluates the relevance of that evidence to the correction.

Original Defect Resolved ≠ No New Defects Introduced.

Verification Evidence Governance

DFM shall preserve the evidence supporting material resolution claims.

Evidence may include:

• Test execution records

• Corrected behavior observations

• Artifact identity

• Configuration correspondence

• Environment records

• Relevant logs

• Regression assessment references

• Competent technical review

Evidence shall remain associated with the specific defect and correction.

Verification Evidence Present ≠ Verification Evidence Sufficient.

Verification Evidence Competence

DFM shall assess whether evidence is suitable to support the claimed resolution.

Competence may depend on:

• Correct artifact

• Relevant configuration

• Suitable verification method

• Applicable expected behavior

• Actual observations

• Adequate assessment scope

• Material limitations

A successful test against an unrelated code path shall not establish

resolution.

Passing Test Near a Correction ≠ Verified Correction Effectiveness.

Verification Evidence Integrity

DFM shall preserve the integrity of material defect verification evidence.

Relevant considerations may include:

• Evidence source

• Execution identity

• Content integrity

• Artifact correspondence

• Temporal relevance

• Verification record preservation

• Material contradictions

Evidence integrity shall remain distinguishable from evidence competence.

Intact Verification Record ≠ Competent Resolution Evidence.

Defect Closure Governance

DFM shall distinguish administrative closure from technical resolution.

Closure may represent:

• Verified resolution

• Accepted duplicate

• Invalid report

• Superseded defect

• Withdrawn report

• Explicitly deferred defect

• Competently accepted residual defect condition

The closure reason and authority shall remain identifiable.

A defect shall not be described as technically resolved merely because its

administrative record has been closed.

Closed Defect Ticket ≠ Resolved Software Defect.

Conditional Defect Closure

DFM shall support explicitly bounded closure conditions where appropriate.

Conditions may include:

• Resolution verified only for a defined configuration

• Affected functionality withdrawn

• Defect deferred under competent authority

• Workaround established but underlying defect unresolved

• Known residual limitations

Conditional closure shall preserve the unresolved technical condition where applicable.

Conditional Closure ≠ Unconditional Resolution.

Defect Reopening

DFM shall recognize circumstances requiring a defect to be reopened.

Triggers may include:

• Reappearance of the deviation

• Failed resolution verification

• Newly identified affected configurations

• Contradictory evidence

• Incorrect correction attribution

• Incomplete implementation

• Relevant regression findings

Reopening shall preserve historical closure and verification records.

Reopened Defect ≠ Historical Evidence Erasure.

Residual Defect Conditions

DFM shall identify known defect conditions that remain after corrective action or verification.

Residual conditions may include:

• Partial correction

• Unresolved environments

• Known workarounds

• Remaining affected versions

• Unverified edge cases

• Deferred correction

• Known intermittent behavior

Residual defects shall remain visible to QCAM.

Partial Defect Resolution ≠ Complete Defect Resolution.

Known Defect Limitations

DFM shall preserve limitations affecting defect assessment and resolution

claims.

Limitations may include:

• Restricted reproduction scope

• Incomplete environment access

• Missing historical evidence

• Limited test coverage

• Unknown root cause

• Incomplete artifact correspondence

• Unverified configurations

Limitations shall remain connected to the relevant defect governance state.

Bounded Resolution Evidence ≠ Universal Resolution Evidence.

Defect Uncertainty

DFM shall preserve material uncertainty concerning defect recognition,

scope, impact, cause, correction, or resolution.

Uncertainty may arise where:

• Observations are incomplete

• Reproduction is inconsistent

• Artifact identity is unresolved

• Root-cause evidence is insufficient

• Verification conditions are incomplete

• Corrective effects are uncertain

Uncertainty shall not be silently converted into defect resolution.

Unresolved Defect Evidence ≠ Verified Defect Absence.

Defect Conflict

A Defect Conflict exists where material defect evidence or conclusions are incompatible.

Examples may include:

• Contradictory reproduction results

• Conflicting root-cause findings

• Incompatible artifact references

• Different verification outcomes

• Contradictory severity assessments

• Conflicting correction claims

DFM shall preserve the conflict and its supporting evidence.

Conflicting Defect Evidence ≠ Permission to Select the Preferred

Conclusion.

Defect Evidence Gap

A Defect Evidence Gap exists where material information required for competent defect assessment remains unavailable or insufficient.

Examples may include:

• Missing execution logs

• Unidentified affected artifact

• Incomplete reproduction conditions

• Missing correction reference

• Absent verification evidence

• Unresolved environment correspondence

A defect evidence gap shall not automatically establish or disprove the

defect.

Defect Evidence Gap ≠ Proof of Defect Absence.

Critical Defect Gap

A Critical Defect Gap exists where missing or unresolved evidence prevents competent determination of a materially consequential defect condition.

Examples may include:

• A critical calculation defect without verified correction

• A recovery defect whose affected configurations remain unknown

• A high-impact integration defect without suitable reproduction

• A defect marked resolved without verification against the intended

evidence. release artifact.

DFM shall preserve the critical gap for competent downstream governance.

It does not independently authorize release rejection, operational shutdown, or legal liability findings.

Critical Defect Gap ≠ Automatic Software Failure Determination.

Defect History Governance

DFM shall preserve the material history of defect governance objects.

History may include:

• Initial report

• Recognition

• Confirmation

• Classification

• Severity assessment

• Root-cause investigation

• Correction proposals

• Implemented corrections

• Verification

• Closure

• Reopening

• Reassessment

Historical records shall remain distinguishable from current defect states.

Current Defect State ≠ Complete Defect History.

Defect Continuity

DFM shall preserve defect relationships across software versions,

configurations, corrections, and lifecycle transitions.

Continuity may involve:

• Original defect identity

• Affected artifacts

• Related defects

• Correction references

• Verification history

• Reopened defects

• Residual conditions

• Superseded defect records

A defect record shall not lose its governance identity merely because its

software context changes.

Defect Record Migration ≠ Defect Resolution.

Defect Reassessment

DFM shall support reassessment when material defect conditions change.

Triggers may include:

• New software versions

• Configuration changes

• Dependency updates

• New reproduction evidence

• Revised severity

• New operational impact

• Failed verification

• Defect recurrence

• Contradictory evidence

Reassessment shall preserve relevant historical determinations.

Defect Reassessment ≠ Historical Defect Record Erasure.

Defect Governance States

DFM shall support identifiable governance states.

Possible states include:

Reported: A potential defect has been reported.

Under Assessment: The reported deviation is being evaluated.

Unconfirmed: Available evidence does not yet establish the defect.

Confirmed: Competent evidence supports defect recognition.

Classification-Pending: The defect type remains unresolved.

Impact-Pending: Material impact remains insufficiently assessed.

Investigation-Active: Root-cause or scope investigation is underway.

Correction-Proposed: A corrective action has been proposed.

Correction-Pending: An approved correction has not yet been implemented.

Correction-Implemented: An identifiable corrective change has been made.

Verification-Pending: Competent resolution verification remains

outstanding.

Verification-Failed: Available verification evidence does not support the claimed resolution.

Resolution-Verified: Competent evidence supports resolution within the assessed scope.

Conditionally-Resolved: Resolution is supported only within explicitly bounded conditions.

Residual-Defect: A known defect condition remains.

Deferred: Correction or resolution has been postponed under competent

authority.

Closed: The administrative defect record has been closed with an

identifiable reason.

Reopened: A previously closed or resolved defect requires renewed

assessment.

Conflict-Detected: Material defect evidence or conclusions are incompatible.

Evidence-Limited: Available evidence is insufficient.

Critical-Gap: Missing evidence prevents competent determination of a

materially consequential condition.

Superseded: A later defect governance record has replaced the earlier

reference.

Unresolved: Available information does not support a sufficiently determined state.

Administrative, technical, and evidentiary states shall remain

distinguishable.

A defect may be administratively closed while its underlying technical condition remains unresolved.

Administrative Defect State ≠ Technical Defect State.

Governed Software Defect State

The principal output of DFM is:

Governed Software Defect State

The state shall preserve sufficient information to identify:

Defect Governance Object: The governed defect assessment.

Defect Identity: The identifiable defect reference.

Report Origin: The source of the report.

Defect Observation: The observed anomaly.

Expected Behavior: The applicable expected condition.

Observed Behavior: The actual behavior recorded.

Expected–Observed Deviation: The recognized discrepancy.

Defect Recognition: The determination supporting defect identification.

Defect Confirmation: The evidence supporting confirmation.

Defect Classification: The recognized defect type.

Defect Scope: The known affected conditions.

Affected Software Object: The relevant software.

Artifact Identity: The affected artifact.

Artifact Version: The relevant software version.

Configuration Correspondence: The applicable configuration.

Environment Correspondence: The relevant operating conditions.

Reproduction Conditions: The conditions under which the defect can be reproduced.

Reproducibility: The recognized reproduction state.

Intermittent Behavior: Relevant inconsistent occurrences.

Severity: The technical significance.

Priority: The recognized correction urgency.

Impact: The actual or supported consequences.

Criticality: The material significance.

Exposure: The conditions affecting defect occurrence.

Propagation: The known or potential downstream effects.

Root-Cause Investigation: The investigation record.

Root-Cause Evidence: The evidence supporting causal findings.

Defect Attribution: The supported technical attribution.

Dependency Relationships: Relevant component or service relationships.

Ownership References: Relevant assigned functions.

Correction Proposal: The proposed corrective action.

Correction Authority References: The applicable authorization conditions.

Correction Implementation: The implemented corrective change.

Corrected Artifact: The artifact containing the correction.

Resolution Criteria: The conditions required for verified resolution.

Resolution Verification: The verification determination.

Verification Method Suitability: The suitability of the method.

Verification Artifact: The software actually verified.

Verification Configuration: The relevant configuration.

Verification Environment: The relevant environment.

Regression Verification: Relevant regression evidence.

Verification Evidence: The evidence supporting resolution.

Evidence Competence: The suitability of that evidence.

Evidence Integrity: The condition of supporting evidence.

Closure State: The administrative closure condition.

Conditional Closure: Relevant bounded closure conditions.

Reopening History: Material reopened defect events.

Residual Defect Conditions: Known remaining defects.

Known Limitations: Relevant assessment restrictions.

Defect Uncertainty: Material unresolved conditions.

Defect Conflicts: Contradictory evidence or conclusions.

Evidence Gaps: Missing material evidence.

Critical Defect Gaps: Material unresolved conditions requiring competent

attention.

Defect History: The historical governance record.

Defect Continuity: Preservation of lifecycle relationships.

Defect Reassessment: Material reassessment history.

Defect Governance State: The recognized defect condition.

Downstream References: Information required by QCAM and parent SQS

governance.

The state shall not independently establish overall software quality

acceptance, release authorization, deployment authorization, legal compliance, or legal liability.

Minimum Implementation Framework

A conforming implementation of DFM must provide the following six functional capabilities.

• Defect Object, Observation, Recognition, Classification, and Software

Correspondence Governance

Capability to establish identifiable defect governance objects, preserve

report origins, expected and observed behavior, identify material deviations, recognize and confirm defects, classify defect types, and associate findings with affected software artifacts, versions, configurations, and environments.

The implementation must distinguish defect reports and failed tests from sufficiently confirmed software defects.

• Defect Scope, Reproduction, Severity, Priority, Impact, Criticality, and

Exposure Governance

Capability to establish defect scope, preserve reproduction conditions,

recognize intermittent behavior, assess severity, priority, impact, criticality, exposure, and potential propagation.

The implementation must distinguish technical severity from correction priority and potential consequences from demonstrated operational impact.

• Root-Cause, Attribution, Dependency, Ownership, and Correction Reference

Governance

Capability to preserve root-cause investigations, assess supporting

evidence, establish bounded technical attribution, identify dependency relationships, preserve relevant responsibility references, and associate proposed or implemented corrections with competent authority references.

The implementation must distinguish plausible root causes from verified causal findings and implemented corrections from verified resolution.

• Resolution Criteria, Verification, Artifact Correspondence, Regression,

and Evidence Governance

Capability to establish defect resolution criteria, verify corrections

against relevant artifacts, configurations, and environments, evaluate verification method suitability, preserve regression references, and assess verification evidence competence and integrity.

The implementation must distinguish passing verification activities from sufficient evidence that the relevant defect has been resolved.

• Defect Closure, Residual Conditions, Conflict, Uncertainty, Gap,

History, and Continuity Governance

Capability to distinguish administrative closure from technical resolution,

preserve conditional closure, residual defects, reopening, evidence conflicts, uncertainty, gaps, critical gaps, historical records, lifecycle continuity, and reassessment.

The implementation must not erase unresolved technical conditions merely because a defect ticket has been closed.

• Governed Software Defect State and SQS Handoff Governance

Capability to produce a Governed Software Defect State with sufficient

defect identity, observation, recognition, classification, affected artifact, severity, impact, correction references, resolution verification, evidence, residual limitations, uncertainty, and continuity information for competent QCAM and parent SQS governance.

The implementation must not represent administrative closure, correction implementation, or a favorable test report as independently sufficient proof of defect resolution. GIE™ Compatibility

DFM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Defect governance object identification

• Defect report origin analysis

• Expected–observed behavior comparison

• Defect recognition assessment

• Defect classification

• Defect scope analysis

• Artifact and version correspondence

• Configuration correspondence analysis

• Environment correspondence analysis

• Reproduction condition identification

• Intermittent defect analysis

• Defect severity assessment

• Defect priority analysis

• Defect impact assessment

• Defect criticality assessment

• Exposure analysis

• Potential propagation mapping

• Root-cause investigation support

• Root-cause evidence assessment

• Technical attribution analysis

• Dependency relationship mapping

• Ownership reference analysis

• Correction proposal evaluation

• Correction authority reference assessment

• Correction implementation mapping

• Corrected artifact correspondence

• Resolution criteria analysis

• Verification method suitability assessment

• Resolution verification evidence analysis

• Regression verification assessment

• Verification evidence competence assessment

• Verification evidence integrity assessment

• Closure state analysis

• Conditional closure assessment

• Residual defect identification

• Defect reopening analysis

• Defect uncertainty recognition

• Defect conflict detection

• Evidence gap detection

• Critical defect gap detection

• Defect history reconstruction

• Defect continuity mapping

• Defect reassessment support

• Governed software defect state representation

GIE™ may identify candidate defects, propose root-cause hypotheses, detect inconsistent correction records, or identify missing verification evidence.

Such outputs shall remain distinguishable from confirmed defects, verified

root causes, authorized corrections, and demonstrated resolution.

GIE™ must not fabricate defect observations, invent verification evidence, infer root-cause certainty from correlation, treat a closed issue ticket as proof of resolution, or perform unauthorized software changes under the pretext of defect correction.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical DFM governance logic is:

Receive Governed Software Quality Requirements State

↓

Receive Governed Quality Criteria Definition State

↓

Receive Governed Software Testing & Coverage State

↓

Identify Defect Governance Object

↓

Preserve Defect Report Origin

↓

Identify Expected and Observed Behavior

↓

Assess Expected–Observed Deviation

↓

Determine Defect Recognition and Confirmation

↓

Classify Defect

↓

Establish Defect Scope

↓

Identify Affected Software Object and Artifact

↓

Establish Version, Configuration, and Environment Correspondence

↓

Preserve Reproduction Conditions

↓

Assess Reproducibility and Intermittent Behavior

↓

Assess Severity, Priority, Impact, and Criticality

↓

Assess Exposure and Potential Propagation

↓

Investigate Root Cause

↓

Assess Root-Cause Evidence

↓

Establish Bounded Technical Attribution

↓

Identify Dependencies and Ownership References

↓

Identify Proposed Correction

↓

Preserve Correction Authority References

↓

Identify Implemented Correction

↓

Establish Corrected Artifact Correspondence

↓

Define Resolution Criteria

↓

Assess Verification Method Suitability

↓

Verify Against Relevant Artifact, Configuration, and Environment

↓

Assess Regression Evidence

↓

Preserve Verification Evidence

↓

Assess Verification Evidence Competence and Integrity

↓

Determine Technical Resolution State

↓

Identify Administrative Closure Conditions

↓

Preserve Residual Defect Conditions

↓

Identify Defect Uncertainty, Conflicts, and Evidence Gaps

↓

Determine Defect Governance State

↓

Establish Governed Software Defect State

↓

Handoff to QCAM and Parent SQS Governance

↓

Preserve Defect History and Continuity

↓

Reassess When Material Defect Conditions Change

The principal output is:

Governed Software Defect State

Governance Outputs

DFM may produce:

Defect Governance Object

Defect Identity Record

Defect Report Origin Record

Defect Observation Record

Expected Behavior Reference

Observed Behavior Record

Expected–Observed Deviation Assessment

Defect Recognition Assessment

Defect Confirmation Record

Defect Classification Record

Defect Scope Record

Affected Software Object Record

Defective Artifact Identity Record

Defective Artifact Version Record

Configuration Correspondence Record

Environment Correspondence Record

Defect Reproduction Conditions Record

Defect Reproducibility Assessment

Intermittent Defect Record

Defect Severity Assessment

Defect Priority Record

Defect Impact Assessment

Defect Criticality Assessment

Defect Exposure Assessment

Defect Propagation Assessment

Root-Cause Investigation Record

Root-Cause Evidence Assessment

Defect Attribution Record

Defect Dependency Record

Defect Ownership Reference

Defect Correction Proposal

Correction Authority Reference

Correction Implementation Record

Corrected Artifact Correspondence Record

Defect Resolution Criteria Record

Defect Resolution Verification Record

Verification Method Suitability Assessment

Verification Artifact Identity Record

Verification Configuration Record

Verification Environment Record

Regression Verification Record

Verification Evidence Register

Verification Evidence Competence Assessment

Verification Evidence Integrity Assessment

Defect Closure Record

Conditional Defect Closure Record

Defect Reopening Record

Residual Defect Record

Known Defect Limitation Record

Defect Uncertainty State

Defect Conflict

Defect Evidence Gap

Critical Defect Gap

Defect History Record

Defect Continuity Record

Defect Reassessment Record

Defect Governance State

SQS Defect Governance Handoff Record

The principal output is:

Governed Software Defect State

Critical Distinctions

DFM preserves the following distinctions:

Defect Closure ≠ Verified Defect Resolution

Defect Report ≠ Confirmed Software Defect

Defect Report Source ≠ Defect Validity

Observed Software Anomaly ≠ Confirmed Software Defect

Unexpected Behavior ≠ Defect Without an Applicable Expected Condition

Reported Behavior ≠ Independently Verified Behavior

Observed Deviation ≠ Established Root Cause

Failed Test ≠ Confirmed Software Defect

Non-Reproducible Defect ≠ Nonexistent Defect

Defect Classification ≠ Defect Severity

Observed Defect Scope ≠ Complete Defect Scope

Affected Software Component ≠ Root-Cause Component

Declared Defective Version ≠ Verified Defective Artifact

Defect in Version A ≠ Proven Defect in Version B

Defect Resolved in One Configuration ≠ Defect Resolved in Every

Configuration

Environment-Specific Defect ≠ Universal Software Defect

Documented Reproduction Steps ≠ Successfully Reproduced Defect

Failure to Reproduce ≠ Proof of Defect Absence

Intermittent Failure ≠ Automatically Invalid Defect Evidence

Defect Severity ≠ Defect Priority

High Defect Priority ≠ High Technical Severity

Potential Defect Impact ≠ Observed Defect Impact

Critical Defect Classification ≠ Automatic Release Rejection

Defect Exposure Estimate ≠ Verified Defect Occurrence Frequency

Potential Defect Propagation ≠ Demonstrated Propagation

Plausible Root Cause ≠ Verified Root Cause

Correlation with Defect Disappearance ≠ Complete Root-Cause Proof

Technical Defect Attribution ≠ Legal Liability Attribution

Dependency Associated with Defect ≠ Dependency Proven as Root Cause

Assigned Defect Owner ≠ Demonstrated Defect Resolution Capability

Proposed Defect Correction ≠ Implemented Defect Correction

Authority to Diagnose a Defect ≠ Authority to Modify Production Software

Correction Implemented ≠ Defect Resolved

Correction in Source Code ≠ Correction in Relevant Executable Artifact

Defined Resolution Criteria ≠ Demonstrated Resolution

Successful Verification Method Execution ≠ Suitable Resolution

Verification

Verification Against an Unidentified Artifact ≠ Verified Defect Resolution

Verified Correction in One Configuration ≠ Universal Defect Resolution

Verification Environment Similarity ≠ Verified Operational Equivalence

Original Defect Resolved ≠ No New Defects Introduced

Verification Evidence Present ≠ Verification Evidence Sufficient

Passing Test Near a Correction ≠ Verified Correction Effectiveness

Intact Verification Record ≠ Competent Resolution Evidence

Closed Defect Ticket ≠ Resolved Software Defect

Conditional Closure ≠ Unconditional Resolution

Reopened Defect ≠ Historical Evidence Erasure

Partial Defect Resolution ≠ Complete Defect Resolution

Bounded Resolution Evidence ≠ Universal Resolution Evidence

Unresolved Defect Evidence ≠ Verified Defect Absence

Conflicting Defect Evidence ≠ Permission to Select the Preferred

Conclusion

Defect Evidence Gap ≠ Proof of Defect Absence

Critical Defect Gap ≠ Automatic Software Failure Determination

Current Defect State ≠ Complete Defect History

Defect Record Migration ≠ Defect Resolution

Defect Reassessment ≠ Historical Defect Record Erasure

Administrative Defect State ≠ Technical Defect State

Cross-Architecture Boundary

DFM governs software-quality-specific defect identity, observation,

recognition, classification, severity, impact, technical attribution, correction references, resolution verification, residual conditions, and continuity within SQS and SLGA™.

It does not independently establish software quality requirements, define assessment criteria, execute testing programs, authorize software modifications, determine complete software quality, accept residual quality limitations, certify software security, authorize release, or authorize deployment.

Within SQS:

QRM — Quality Requirements Module governs software quality requirement

establishment.

QCDM — Quality Criteria Definition Module governs quality criterion

definition, applicability, assessment methods, and evidence requirements.

TCMM — Testing & Coverage Management Module governs testing activities,

execution evidence, observed results, and coverage.

DFM — Defect Management Module governs defect identification,

classification, impact, correction references, resolution verification, and continuity.

QCAM — Quality Conformity & Acceptance Module governs quality conformity

assessment, intended-use validation, acceptance authority, acceptance decisions, and residual limitations.

Within SLGA™:

Software Requirements Standard (SRS) governs the broader software

requirements lifecycle.

Software Creation Standard (SCS) governs software creation.

Software Dependency Standard (SDS) governs software dependencies.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs software release.

Software Deployment Standard (SDPS) governs software deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

changes.

Software Retirement Standard (SRTS) governs software retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Relevant responsibility

and accountability relationships.

ORA™ — Operational Reality Architecture: Competent assessment of actual

software behavior and operating conditions.

ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

defect investigation and correction actions.

AGI™ — Agent Governance Infrastructure: Relevant agent permissions,

tool-path authority, and constituent action governance.

DFM preserves software-defect-specific governance without replacing these

competent domains.

Canonical Boundary

DFM begins when a reported or observed software deviation requires recognition, assessment, investigation, correction reference, resolution verification, or reassessment.

DFM governs:

• Defect governance objects

• Defect identity

• Defect report origins

- Defect observations
- Expected behavior
- Observed behavior
- Expected–observed deviations
- Defect recognition
- Defect confirmation
- Defect classification
- Defect scope
- Affected software objects
- Artifact identity
- Artifact versions
- Configuration correspondence
- Environment correspondence
- Reproduction conditions
- Reproducibility
- Intermittent defects
- Defect severity
- Defect priority
- Defect impact
- Defect criticality
- Defect exposure
- Defect propagation
- Root-cause investigations
- Root-cause evidence
- Technical defect attribution
- Defect dependencies
- Ownership references
- Correction proposals
- Correction authority references
- Correction implementation references
- Corrected artifact correspondence
- Defect resolution criteria
- Resolution verification
- Verification method suitability
- Verification artifact identity
- Verification configuration

• Verification environment

• Regression verification

• Verification evidence

• Verification evidence competence

• Verification evidence integrity

• Defect closure

• Conditional closure

• Defect reopening

• Residual defect conditions

• Known defect limitations

• Defect uncertainty

• Defect conflicts

• Defect evidence gaps

• Critical defect gaps

• Defect history

• Defect continuity

• Defect reassessment

• Defect governance states

DFM does not independently determine:

• Complete software requirements validity

• Software creation authorization

• Dependency suitability

• Configuration baseline establishment

• Quality criterion establishment

• Overall testing sufficiency

• Complete software correctness

• Software change authorization

• Quality acceptance

• Software security

• Release authorization

• Deployment authorization

• Legal compliance

• Legal liability

DFM ends when material defect conditions have been represented as a:

Governed Software Defect State

with sufficient defect identity, observation, recognition, classification,

affected artifact, severity, impact, correction references, resolution verification, evidence, residual limitations, uncertainty, and continuity information for competent downstream QCAM and parent SQS governance.

• Use Case 1 — A Critical Financial Calculation Defect Is Marked Resolved

Without Verification Against the Release Candidate

An organization develops software for processing financial transactions.

During testing, a defect is identified that produces incorrect results when specific transaction values are combined.

The development team:

• Creates a defect ticket

• Identifies a likely source-code error

• Implements a correction

• Runs a unit test

• Marks the defect as resolved

The automated issue-tracking system reports:

Status: Closed

However, the correction was verified only in a development build.

The release candidate was generated from a different build process and

contains additional configuration changes.

No competent evidence establishes that the correction is present and effective in the actual release candidate.

DFM evaluates:

Defect Observation → Defect Recognition → Affected Artifact → Correction →

Corrected Artifact → Resolution Criteria → Verification Evidence → Defect State

The assessment identifies:

• A confirmed software defect

• An identifiable corrective code change

• A successful test against a development artifact

• Missing verification against the intended release candidate

• Unresolved configuration correspondence

• Insufficient evidence supporting technical resolution in the release

candidate.

The relevant governance state may be:

Verification-Pending

or:

Evidence-Limited

despite the administrative ticket being closed.

The critical distinctions are:

Correction Implemented ≠ Defect Resolved.

Correction in Source Code ≠ Correction in Relevant Executable Artifact.

Closed Defect Ticket ≠ Resolved Software Defect.

Defect Closure ≠ Verified Defect Resolution.

The result: DFM prevents administrative defect closure from being

interpreted as proof that the relevant software artifact no longer contains the defect.

Use Case 2 — An Autonomous Development Agent Claims to Have Fixed an

Intermittent Production Defect

An organization uses an autonomous software development agent to

investigate intermittent failures in a distributed application.

The failures occur during periods of high concurrency.

The agent analyzes logs and identifies a possible race condition.

It modifies synchronization logic and executes a small local test suite.

The tests pass.

The agent reports:

"Root cause identified and defect resolved."

However:

• The original failure was intermittent

• The local tests did not reproduce production concurrency

• The agent's root-cause hypothesis was not independently verified

• The modified artifact was not evaluated under representative workload

• No competent evidence establishes that the production failure

condition has been eliminated.

DFM evaluates:

Observed Failure → Root-Cause Hypothesis → Technical Attribution →

Correction → Verification Method → Operational Correspondence → Resolution Evidence

The assessment identifies:

• A sufficiently supported defect observation

• A plausible but unverified root-cause hypothesis

• An implemented corrective change

• Limited verification evidence

• Unresolved operational correspondence

• Insufficient evidence supporting complete resolution

The relevant governance state may be:

Correction-Implemented

with:

Verification-Pending

or:

Evidence-Limited

The critical distinctions are:

Plausible Root Cause ≠ Verified Root Cause.

Correlation with Defect Disappearance ≠ Complete Root-Cause Proof.

Successful Verification Method Execution ≠ Suitable Resolution

Verification.

Correction Implemented ≠ Defect Resolved.

The result: DFM prevents an autonomous development agent from converting

a plausible diagnosis and successful local tests into an unsupported claim of verified production defect resolution.

Canonical Closing Statement

Software defects are not governed merely because they are reported, classified, assigned, corrected, or marked as closed.

A failed test may indicate a software defect, but it may also reflect an invalid test, unsuitable environment, or incorrect expected result.

A plausible root cause may explain an observed failure without being sufficiently verified.

A corrective code change may exist without being included in the software artifact that matters.

A verification test may pass while failing to reproduce the original defect conditions.

And an administrative ticket may be closed while the underlying technical defect remains unresolved.

Defect Report ≠ Confirmed Software Defect.

Plausible Root Cause ≠ Verified Root Cause.

Correction Implemented ≠ Defect Resolved.

Closed Defect Ticket ≠ Resolved Software Defect.

Defect Closure ≠ Verified Defect Resolution.

These distinctions become increasingly important as AI systems and

autonomous development agents identify defects, propose corrections, execute tests, and report successful remediation.

The ability to generate a correction does not establish that the correction is effective.

The ability to execute a test does not establish that the test is suitable.

And the ability to close an issue does not establish that the underlying software behavior has been corrected.

DFM therefore governs the evidentiary relationship between defect

recognition, affected software state, technical investigation, corrective action, and verified resolution.

It does not require every defect to be immediately corrected.

It requires unresolved defects, deferred corrections, partial resolutions,

verification limitations, and residual conditions to remain explicit.

It does not equate defect management with defect elimination.

It establishes the conditions under which defect claims and resolution

claims can be competently recognized.

DFM transforms software defect management from a sequence of

issue-tracking statuses, correction activities, and automated reports into a governed evidentiary process — ensuring that software organizations can establish not merely whether a defect was reported or a correction was implemented, but what behavior was actually defective, which software state was affected, what evidence supports the diagnosis, whether the relevant correction was verified, and what limitations remain.

A software defect is not governed because it has been assigned, corrected, or closed. It is governed when its identity, observed behavior, affected software state, significance, correction references, verification evidence, residual limitations, and lifecycle continuity remain sufficiently established for competent software quality governance.